

Document number	R3211R0
Date	2024-03-31
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::transform_join

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes the Tier 1 adaptor in [P2760](#): `views::transform_join` to improve the C++26 ranges facilities, which is a simple composition of `views::transform` and `views::join`.

Revision history

R0

Initial revision.

Discussion

`views::transform_join`, called `flat_map` in most languages, is a commonly-used operation on data streams. It transforms the input elements into an output sequence through the transformation function, and flattens the resulting sequence of the sequence.

Benefiting from the enhancement of `join_view` in [P2328](#), `views::transform_join(E, F)` is now completely equivalent to `views::join(views::transform(E, F))`. We no longer need to use `views::cache_last` ([P3138](#)) to cache the prvalue range returned by the transform function, `join_view` already do this internally for us.

Given that [P2760](#) considers this adaptor a high priority, the commonness of such operation in other languages, and the convenience brought by reducing the user's typing, the author believes that it is time to bring `views::transform_join` into the standard.

After this, the famous Pythagorean triples [formula in range/v3](#) now can be written as:

```
// Define an infinite range containing all the Pythagorean triples:
auto triples = views::iota(1) | views::transform_join([](int z) {
    return views::iota(1, z + 1) | views::transform_join([](int x) {
        return views::iota(x, z + 1) | views::transform_join([](int y) {
            return views::repeat(tuple(x, y, z), x * x + y * y == z * z ? 1 : 0);
        });
    });
});

// Display the first 100 triples
for (auto triple : triples | views::take(100))
    println("{} ", triple);
```

Design

Should we name it `views::flat_map`?

Nope.

Although most other languages call this operation `flat_map`, given that the standard already has a container of the same name (i.e. `std::flat_map`) with a completely different meaning, the author believes that `views::transform_join` is a more intuitive and standard-

compliant name as it is the composition of `views::transform` and `views::join`.

It's worth noting that `range/v3` calls it `views::for_each`, which might be a [more confusing name](#) so is not considered here.

Should we introduce a new `transform_join_view` class?

Nope.

As [stated in P2760](#): "Importantly, there really isn't much benefit to providing a bespoke `transform_join` as opposed to simply implementing it in terms of these two existing adaptors." The author holds the same view.

Although this will bring an adaptor that combines *two* existing adaptors to the standard for the first time, the author believes that such change is still worthwhile.

First of all, the cost of creating a new `transform_join_view` class and associated iterator is quite high because `join_view` is already a complex view. Especially when its functionality can be completely synthesized by simply combining two adaptors, introducing new classes has no observable benefit except bringing unnecessary complexity and redundancy.

Second, by combining two adaptors, `views::transform_join` can automatically inherit their previous and subsequent enhancements, such as support for stashing iterators ([P2770](#)) and extension for conditionally borrowed ([P3117](#), hope so?), because `views::transform_join` is essentially a specialized `join_view`.

In summary, the authors stick with the current design. Note that the design is also consistent with the [implementation in range/v3](#).

Should we spell it `views::join(views::transform(E, F))` or `join_view(transform_view(E, F))`?

As currently worded, `views::transform(E, F)` is expression-equivalent to `transform_view(E, F)` and `views::join(E)` is expression-equivalent to `join_view<views::all_t<decltype((E))>>{E}`. Since the latter's `E` is always `transform_view`, this causes `views::all_t` to have no effect.

That is to say, `views::join(views::transform(E, F))` is exactly `join_view(transform_view(E, F))`. In this case, the author prefers to use the more primitive form of the latter as the wording.

Implementation experience

The author implemented `views::transform_join` based on `libc++`. The details are relatively simple in less than 20 lines, see [here](#).

Proposed change

This wording is relative to [N4971](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_ranges_transform_join 2024XXL // freestanding, also in <ranges>
```

2. Modify 26.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
#include <compare>           // see [compare.syn]
#include <initializer_list>   // see [initializer.list.syn]
#include <iterator>           // see [iterator.synopsis]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified join_with = unspecified; } // freestanding

    // [range.transform.join], transform join view
    namespace views { inline constexpr unspecified transform_join = unspecified; } // freestanding
    [...]
}
```

3. Add 26.7.2 **Transform join view** [\[range.transform.join\]](#) after 26.7.15 [\[range.join.with\]](#) as indicated:

-1- A transform-join view flattens the result of an underlying sequence after applying a transformation function that returns another range to each element.

-2- The name `views::transform_join` denotes a range adaptor object ([\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::transform_join(E, F)` is expression-equivalent to `join_view(transform_view(E, F))`.

-3- [Example 1:

```
vector ints{0, 1, 2};  
for (auto&& elem : ints | views::transform_join([](int i) { return views::repeat(i, 3); })))  
    cout << elem << ' '; // prints 0 0 0 1 1 1 2 2 2
```

— end example]

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>

[P2328R1]

Tim Song. join_view should join all views of ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2328r1.html>

[P3138R0]

Tim Song. views::cache_last. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3138r0.html>

[P2770R0]

Tim Song. Stashing stashing iterators for proper flattening. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2770r0.html>

[P3117R0]

Zach Laine. Extending Conditionally Borrowed. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3117r0.html>